Technical report — SmartStart: a rubric-grounded Apertus interview coach

- **Track:** Track 2A — FHGR: AI-Powered Job Interview Coach
- **Event:** Online (1–16 October 2026)
- **Team:** SmartStart — Chu Jia, Sice Huang (University of Zurich)
- **Demo:** `make run` → web app on `http://localhost:7860` + Aegra API on `http://localhost:2026` (graph `agent`); 2-minute video: see submission

1. Summary

Young people preparing for their first apprenticeship interview rarely get repeated, individual practice. **SmartStart** is an Apertus-1.5-8B interview coach that runs a realistic, adaptive interview in **German, French or Italian** (Swiss-German dialect input understood) and ends with personal, guideline-compliant feedback. Instead of one large prompt, SmartStart combines (i) a deterministic interview state machine that follows the FHGR interview flow, (ii) **retrieval of FHGR expert-annotated answers as calibration anchors**, (iii) a **two-call loop per answer** (assess → interviewer turn) and (iv) a **rule checker for the FHGR feedback guidelines** that triggers a single rewrite only when needed.

Headline results (Apertus 1.5 8B, vLLM on one GH200 at CSCS): expert-exemplar retrieval raises the accuracy of the coach's next-action decision from **0.44 to 0.83** (100 expert-annotated answers, de/fr/it/Swiss German) and detects all distress cases (recall 1.0 vs. 0.0); quality levels are within ±1 of the expert in **99 %** of cases; all **28 FHGR scenarios** run end-to-end with **2.11 LLM calls per candidate answer** (limit < 5) and 27/28 final feedbacks pass all automatic guideline checks.

2. Architecture

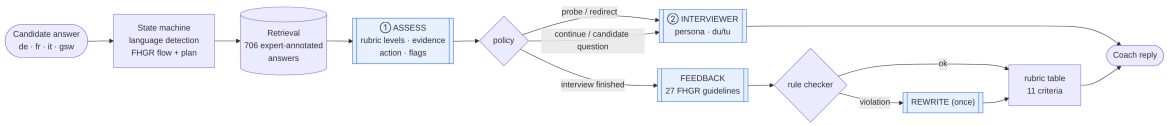


Figure 1 — SmartStart pipeline per candidate answer (blue = Apertus calls).

Component	File	LLM calls
LangGraph graph served by Aegra (`agent`, port 2026); session state persisted per thread in PostgreSQL, Redis job queue	`src/main.py`, `docker-compose.yml`	–
Language & dialect detection (stop-word / Mundart-marker heuristic)	`smartstart/langdetect.py`	0
Interview planner: FHGR flow introduction → motivation → strengths/weaknesses → situational → candidate questions → closing; occupation-specific questions via the posting's `berufsfeld`; conditional questions (Stellwerk, absences) only if the profile justifies them; FHGR scenario plans; free-text occupation wish matched to a posting	`smartstart/planner.py`	0
Exemplar retrieval over 706 expert-annotated answers (char-trigram Jaccard — robust across de/fr/it and dialect spelling; same question first; diverse quality levels; leave-one-out in evaluation)	`smartstart/retrieval.py`	0
Assess: JSON with rubric levels (1–4 / null), evidence quote, strength, improvement, coach action (`continue / probe / redirect /`	`smartstart/coach.py`	**1 per answer**

Component	File	LLM calls
`support_and_refer`), problem flags, follow-up, coaching tip		
Deterministic policy: ≤ 1 follow-up per question, ≤ 4 per interview; candidate questions answered only from posting facts; explicit distress signal ⇒ supportive mode with referral to a real person (L3)	`smartstart/coach.py`	0
Interviewer turn in the persona of the posting's Berufsbildner/in, du/tu form	`smartstart/coach.py`	**1 per answer**
Final feedback (strengths → max. 3 improvements, each with a short quote → one next step → encouragement)	`smartstart/coach.py`	1 per interview
Guideline checker T3, B3, L2, L4, K1, R1/R3, S3, S4 ⇒ **rewrite once** only if violated	`smartstart/rules.py`	0–1 per interview
Deterministic rubric table for all 11 FHGR criteria; "no evidence" instead of guessing (K3)	`smartstart/coach.py`	0

Web app (`src/smartstart/webui.py`, standard library only): the candidate picks a language and a posting and does the interview; a side panel shows the coach's live analysis of every answer (rubric levels, evidence quote, next action, flags). After the feedback, any answer can be **practised again**: the improved answer is re-assessed (1 call) and compared criterion by criterion — the comparison is deterministic, so the coach cannot claim progress the rubric does not show.

Two feedback modes: `end` (realistic interview, feedback at the end — default) and `per_answer` (adds the assessment's one-line tip after each answer, at no extra LLM cost).

What is new. (1) The FHGR annotations are used *at inference time* as few-shot calibration anchors selected per question and per answer — no fine-tuning, yet the coach inherits the experts' judgement of when to probe. (2) Control flow is deterministic and only language generation is delegated to the LLM, which keeps the call budget fixed (2 per answer) and behaviour reproducible. (3) The FHGR feedback guidelines are enforced twice: in the prompt and by a lexical checker that triggers a targeted rewrite.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B` (bf16), the only generative model in the system.
- **How it is used:** inference — structured assessment (JSON), interviewer role-play, feedback generation and rewrite. In the evaluation Apertus also plays the candidate and acts as judge (§5).
- **Where it runs:** vLLM, official image `ghcr.io/swiss-ai/vllm_apertus_1.5_release` (arm64), one NVIDIA GH200 node on CSCS Clariden (project g238); OpenAI-compatible API reached through an SSH tunnel. The coach only needs `LLM_BASE_URL / LLM_API_KEY / LLM_NAME`, so it runs unchanged against any on-premise endpoint (`docker compose --profile gpu` starts vLLM on a single consumer GPU, 8B bf16 ≈ 16 GB). Note: in October 2026 Apertus 1.5 is not yet supported by released MLX / transformers versions; on a MacBook (M4 Pro) the swiss-ai transformers branch loaded the model but MPS bf16 produced corrupted text and fp32 exceeded memory, which is why we served it with vLLM.
- **Prompts:** `src/smartstart/prompts.py`. Instructions in English, all user-facing text in the interview language; temperature 0 for assessment, 0.3–0.4 for generation; fixed seed 42. Every request is a single system + user message (the Apertus 1.5 vLLM chat template rejects replayed multi-turn assistant messages, so dialogue history is passed as text).

4. Data

Only the FHGR/Profolio challenge data (provided for Hack Apertus; `data/fhgr/`, 2.6 MB): 242 questions (+ Swiss-German variants), 706 answers at 4 quality levels in de/fr/it and 8 dialects with expert annotations, the 11-criterion rubric, 27 feedback guidelines, 24 candidate profiles, 30 postings, 28 scenarios with coach success criteria, 30 good-vs-poor feedback pairs. All records are synthetic or didactic; no personal data. No other data was used and no model was trained.

5. Evaluation

The official LLM-as-judge benchmark is hidden, so we built three proxy experiments (`make eval`; raw results in `eval/results/`).

E1 — Assessment vs. FHGR experts. 100 annotated answers (25 each: de, fr, it, Swiss German; stratified by quality level). Leave-one-out: an answer's own annotation is never retrieved.

Setup	Level acc	±1 acc	Level MAE	Action acc	Follow-up acc	Criteria MAE	Distress recall
Baseline (rubric only, no retrieval)	0.59	0.97	0.45	0.44	0.79	0.54	0.00
SmartStart (expert-exemplar retrieval)	**0.65**	**0.99**	**0.36**	**0.83**	**0.87**	**0.46**	**1.00**

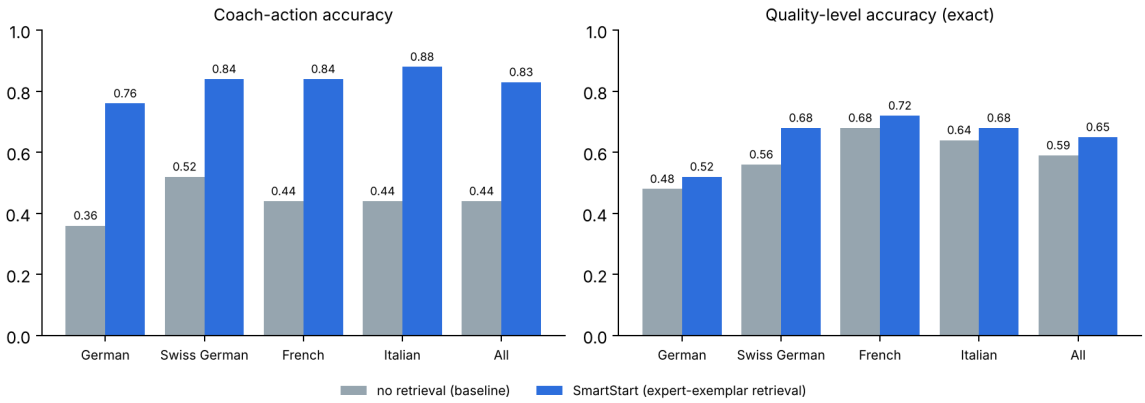


Figure 2 — Agreement with FHGR experts, with and without expert-exemplar retrieval.

Retrieval helps in every language group; the largest effect is on *what to do next* (action accuracy +0.39), which is exactly the decision that makes an interview adaptive. Per language (ours): action accuracy de 0.76, Swiss German 0.84, fr 0.84, it 0.88; level MAE 0.28–0.48.

E2 — 28 simulated interviews. Apertus plays each FHGR scenario candidate (profile, persona, dialect where expected) until the coach's final feedback. All 28 runs completed: 511 candidate answers, 47 adaptive follow-ups, **2.11 LLM calls per answer** (max. 2.17 in any scenario; FHGR limit < 5). The guideline checker flagged 2/28 feedbacks before rewrite and 1/28 after.

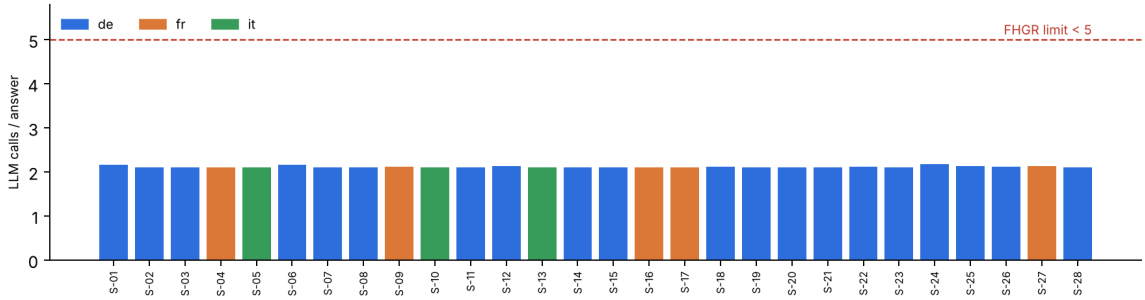


Figure 3 — LLM calls per candidate answer in the 28 FHGR scenarios.

E3 — LLM-as-judge. The judge (Apertus 1.5 8B, temperature 0) is first validated on the 30 FHGR good-vs-poor feedback pairs: it ranks the good feedback higher in **0.90** of 29 valid pairs. On the transcripts it marks **100 %** of the scenario success criteria as met and rates realism, adaptivity, relevance, appropriateness and feedback quality **4.96/5** on average (easy 5.0, medium 5.0, hard 4.8; de 4.95, fr 5.0, it 5.0; language consistency 5.0).

Consistency. Results are stable across languages (action accuracy 0.76–0.88; judge 4.95–5.0), difficulty levels and situations (apprenticeship, online, trial, phone, follow-up, assessment).

6. Limitations

- **Judge ceiling / self-preference:** E3 uses the same model family as the coach and saturates near 5/5; its comments sometimes evaluate the candidate rather than the coach. We therefore treat E1 (agreement with human experts) as the primary evidence and E3 as a sanity check.
- German quality-level accuracy (0.52) is the lowest group; many errors are 3-vs-4 on strong answers.
- Simulated candidates are more articulate than real 15-year-olds; the FHGR data is synthetic and not yet expert-reviewed. No user study yet.
- Text only. The rule checker is lexical; it catches clear violations, not subtle tone problems.
- Dialect is understood on input; output is Standard German (guideline S4).

7. Reproducibility

- `make test` — offline unit tests (mock LLM); `make smoke` — all 28 scenarios with the mock LLM.
- `make run` — Docker: web app (7860) + Aegra API (2026) + PostgreSQL + Redis; Apertus via `LLM_BASE_URL` (`make run-gpu` adds a vLLM container on an NVIDIA GPU).
- `make eval` — E1–E3 + `eval/results/RESULTS.md` (needs an Apertus endpoint).
- CSCS: `scripts/cscs/apertus-vllm.toml` + `serve_apertus.sbatch` reproduce our serving setup.
- Hardware: 1× GH200 (CSCS Clariden) for the model; client on a MacBook Pro M4 Pro. Seeds: `LLM_SEED=42`, sampling seed 13. Runtime: E1 ≈ 4 min, E2 ≈ 45 min, E3 ≈ 5 min.

8. Next steps

- **Speech:** Apertus 1.5 already accepts audio input, so spoken answers can be assessed directly; short plain-text coach turns are ready for a TTS voice (e.g. Piper) and a Swiss-German ASR fallback.
- A teacher dashboard over the per-session rubric levels stored in PostgreSQL.
- LoRA-tuning the assessor on the expert annotations; a study with real classes; an independent judge.

License

Code Apache-2.0; report CC-BY-4.0. FHGR data under the FHGR / Hack Apertus conditions.

AI assistance

Parts of the code and documentation were drafted with the help of an AI assistant (Claude) and reviewed, run and evaluated by the team.

References

- FHGR / Profolio: *Feedbackkriterien zum Vorstellungsgespräch* and challenge datasets, github.com/fhgr/Hackapertus.
- Swiss AI Initiative: Apertus 1.5 model card, huggingface.co/swiss-ai/Apertus-v1.5-8B.
- Kwon et al. (2023): Efficient memory management for LLM serving with PagedAttention (vLLM).